

分治法

1. 寻找凸包 (Graham 扫描法) 2
给定平面上的点集 Q , 使用 Graham 扫描法求 Q 的凸包 (凸包定义: 包含所有点的最小凸多边形, 点或在多边形上或在内部), 要求展示算法的完整步骤。
2. 两个有序数组的中位数 2
设 $X[0:n-1]$ 和 $Y[0:n-1]$ 为两个长度均为 n 的有序数组, 设计 $O(\log n)$ 时间的分治算法, 找出 X 和 Y 的 $2n$ 个元素的中位数 (偶数个元素取中间两个的平均值, 奇数个取最中间元素)。
3. 最大子数组问题 3
给定包含 n 个整数 (有正有负) 的数组 A , 设计 $O(n \log n)$ 时间的分治算法找出和最大的非空连续子数组, 同时设计 $O(n)$ 时间的 Kadane 算法。
4. 循环移位问题 3
给定一个按从小到大排序的数组, 现将其循环右移若干位 (如原数组 $[1,2,3,4,5]$, 循环右移 2 位后为 $[4,5,1,2,3]$), 设计算法计算循环右移的位数 k ($k \geq 0$), 要求时间复杂度尽可能优化。
5. 归并排序 4
给定数组 A , 使用归并排序算法对其进行升序排序, 要求展示分治的”分解 $\rightarrow$ 求解 $\rightarrow$ 合并”完整过程。
6. 折半搜索 (查找成功案例) 4
给定有序数组 A , 使用折半搜索算法查找目标元素 $target$, 要求展示每次查找的 low 、 mid 、 $high$ 值及比较过程, 若查找成功返回元素下标, 失败返回 -1。
7. 折半搜索 (查找失败案例) 5
给定有序数组 A , 使用折半搜索算法查找不存在的目标元素, 重点关注查找失败时的边界调整及终止条件判断。
8. 大整数乘法 (Karatsuba 算法) 5
设 X 和 Y 为两个 n 位大整数 (n 为偶数), X 拆分为 A (前 $n/2$ 位) 和 B (后 $n/2$ 位), Y 拆分为 C (前 $n/2$ 位) 和 D (后 $n/2$ 位), 即 $X = A \cdot 2^{n/2} + B$, $Y = C \cdot 2^{n/2} + D$ 。设计优化方案通过减少乘法次数降低时间复杂度。
9. Strassen 矩阵乘法 6
给定两个 $n \times n$ 矩阵 A 和 B , 使用 Strassen 算法 (分治法) 计算矩阵乘积 $C=A \times B$, 要求展示 7 个中间矩阵的定义及复杂度分析。
10. 快速排序 6
给定数组 A , 使用快速排序算法对其进行升序排序, 要求展示 Partition 过程及完整的递归排序步骤。
11. 第 k 小元素 (选择问题) 7
给定无序数组 A , 使用分治法查找该数组的第 k 小元素, 要求展示 Partition 过程及递归查找步骤。
12. 堆排序 7
给定数组 A , 使用堆排序算法对其进行升序排序, 要求展示最大堆的建立过程、堆顶元素的弹出与堆调整过程。
13. 最近点对问题 8
给定平面上 n 个点的集合 Q , 设计分治算法找出距离最近的两个点, 要求说明预处理、分解、递归求解、合并的完整流程。
14. 天际轮廓问题 8
给定 n 座建筑物, 每个建筑物用三元组 (Li, Ri, Hi) 表示 (Li 为左下 x 坐标, Ri 为右下 x 坐标, Hi 为高度), 设计 $O(n \log n)$ 算法求出这 n 座建筑物的天际轮廓。
15. 两元素和为 X 的分治算法 9
给定由 n 个实数构成的集合 S 和实数 X , 判断 S 中是否存在两个元素的和为 X , 要求设计分治算法求解。
16. 逆序对的分治算法 9
给定实数序列 A , 若 $i < j$ 且 $A[i] > A[j]$, 则 (i, j) 为逆序对。要求用分治方法求序列中逆序对的个数。
17. 数组中两元素和大于 0 的对数量 10
给定大小为 n 的数组 A , 求数组中不同对 (i, j) ($i < j$) 的数量, 使得 $A[i] + A[j] > 0$ 。
18. 分治算法解题的固有方法论 11
分治算法 (Divide and Conquer) 的核心思想是”大事化小, 小事化了, 最后合并”。标准三步走策略: 分解 (Divide)、解决 (Conquer)、合并 (Combine)。复杂度分析使用主定理 $T(n) = aT(n/b) + f(n)$ 。

1. 寻找凸包（Graham 扫描法）

给定平面上的点集 Q ，使用 Graham 扫描法求 Q 的凸包（凸包定义：包含所有点的最小凸多边形，点或在多边形上或在内部），要求展示算法的完整步骤。

- **基准点选择**：找 y 坐标最小的点 p_0 （ y 相同时取 x 最小），保证 p_0 必在凸包上
- **极角排序**：其余点按相对 p_0 的极角从小到大排序，共线点按距离由近及远，用叉积判断避免角度计算
- **栈维护凸性**：从第 3 个点开始扫描，若新点与栈顶两点不构成左转（叉积 ≤ 0 ）则弹出栈顶，直至构成左转或栈只剩 1 个点
- **时间复杂度**：找基准点 $O(n)$ ，极角排序 $O(n\log n)$ ，扫描 $O(n)$ ，总计 $O(n\log n)$

```
int cross(Point p0, Point p1, Point p2) {
    return (p1.x - p0.x) * (p2.y - p0.y) -
           (p1.y - p0.y) * (p2.x - p0.x);
}
int dist2(Point p1, Point p2) {
    return (p1.x - p2.x) * (p1.x - p2.x) +
           (p1.y - p2.y) * (p1.y - p2.y);
}
bool compare(Point p1, Point p2) {
    int c = cross(p0, p1, p2);
    if (c == 0) return dist2(p0, p1) < dist2(p0, p2);
    return c > 0;
}
vector<Point> grahamScan(vector<Point>& points) {
    int n = points.size();
    int p0_idx = 0;
    for (int i = 1; i < n; i++) {
        if (points[i].y < points[p0_idx].y ||
            (points[i].y == points[p0_idx].y &&
             points[i].x < points[p0_idx].x))
            p0_idx = i;
    }
    swap(points[0], points[p0_idx]);
    p0 = points[0];
    sort(points.begin() + 1, points.end(), compare);
    vector<Point> S;
    S.push_back(points[0]);
    S.push_back(points[1]);
    for (int i = 2; i < n; i++) {
        while (S.size() >= 2 &&
               cross(S[S.size()-2], S.back(), points[i]) <= 0) {
            S.pop_back();
        }
        S.push_back(points[i]);
    }
    return S;
}
```

1. 扫描点集找到 y 最小的点作为 p_0 （有多个时取最左边的），把它交换到数组首位
2. 从第二个点开始，按“相对 p_0 的极角”排序：用叉积判断，共线时离 p_0 近的排前面
3. 新建一个栈，先把 p_0 和排序后的第一个点入栈
4. 从第三个点开始，检查栈顶两个点加上当前点是否“左转”：若不是（右转或共线）就弹出栈顶，重复检查直到满足左转或栈只剩一个点
5. 把当前点压入栈，继续处理下一个点
6. 全部处理完后，栈里剩下的点按顺序连起来就是凸包

2. 两个有序数组的中位数

设 $X[0:n-1]$ 和 $Y[0:n-1]$ 为两个长度均为 n 的有序数组，设计 $O(\log n)$ 时间的分治算法，找出 X 和 Y 的 $2n$ 个元素的中位数（偶数个元素取中间两个的平均值，奇数个取最中间元素）。

- **问题转化**： $2n$ 个元素的中位数是第 n 小和第 $n+1$ 小元素的平均值，需同时找到这两个元素
- **分治策略**：对两个数组同时划分，设 X 的划分边界为 i （前 i 个元素）， Y 的划分边界为 j （前 j 个元素），要求 $i+j=n$ （确保左半部分总长度为 n ）
- **终止条件**：若 $X[i-1] \leq Y[j]$ 且 $Y[j-1] \leq X[i]$ ，则左半最大值和右半最小值即为第 n 小和第 $n+1$ 小元素
- **范围缩小**：若 $X[i-1] > Y[j]$ ，说明 X 的前 i 个元素过大，需减小 i ；若 $Y[j-1] > X[i]$ ，说明 Y 的前 j 个元素过大，需减小 j
- **时间复杂度**：每次排除一半元素， $O(\log n)$

```
double findMedianSortedArrays(vector<int>& X, vector<int>& Y) {
    int n = X.size();
    int low = 0, high = n;
    while (low <= high) {
        int i = (low + high) / 2;
        int j = n - i;
        int X_left = (i == 0) ? INT_MIN : X[i - 1];
        int X_right = (i == n) ? INT_MAX : X[i];
        int Y_left = (j == 0) ? INT_MIN : Y[j - 1];
        int Y_right = (j == n) ? INT_MAX : Y[j];
        if (X_left <= Y_right && Y_left <= X_right) {
            if ((n + n) % 2 == 0) {
                return (max(X_left, Y_left) + min(X_right,
                                                    Y_right)) / 2.0;
            } else {
                return min(X_right, Y_right);
            }
        } else if (X_left > Y_right) {
            high = i - 1;
        } else {
            low = i + 1;
        }
    }
    return 0.0;
}
```

1. 在较短的数组上二分划分边界 i （等长时任选一个），计算另一数组的划分边界 $j=n-i$
2. 检查边界条件：若 X 左半最大 $\leq Y$ 右半最小且 Y 左半最大 $\leq X$ 右半最小，说明划分正确
3. 若 X 左半最大 $> Y$ 右半最小，说明 X 的前 i 个元素太大，让 i 减小（向左移）
4. 若 Y 左半最大 $> X$ 右半最小，说明 Y 的前 j 个元素太大，让 i 增大（向右移， j 随之减小）
5. 找到正确划分后，偶数个元素取左半最大和右半最小的平均值，奇数个取右半最小

3. 最大子数组问题

给定包含 n 个整数（有正有负）的数组 A ，设计 $O(n \log n)$ 时间的分治算法找出和最大的非空连续子数组，同时设计 $O(n)$ 时间的 Kadane 算法。

- **分治策略**：将数组 $A[\text{low}..\text{high}]$ 拆分为左半 $A[\text{low}..\text{mid}]$ 和右半 $A[\text{mid}+1..\text{high}]$ ，最大子数组必在左、右、或跨越中点三者之一
- **跨越中点求解**：从 mid 向左扫描得最大左段和，从 $\text{mid}+1$ 向右扫描得最大右段和，两者相加即为跨越中点的最大子数组和
- **递归求解**：左半和右半的最大子数组通过递归求解，终止条件为子数组长度为 1
- **Kadane 算法**：维护 current_max （以当前元素结尾的最大子数组和）和 global_max （全局最大）， $\text{current_max} = \max(A[i], \text{current_max} + A[i])$
- **时间复杂度**：分治法 $O(n \log n)$ ，Kadane 算法 $O(n)$

```
int maxCrossingSubarray(vector<int>& A, int low, int mid, int high) {
    int left_sum = INT_MIN, sum = 0;
    for (int i = mid; i >= low; i--) {
        sum += A[i];
        left_sum = max(left_sum, sum);
    }
    int right_sum = INT_MIN;
    sum = 0;
    for (int i = mid + 1; i <= high; i++) {
        sum += A[i];
        right_sum = max(right_sum, sum);
    }
    return left_sum + right_sum;
}

int maxSubarrayDC(vector<int>& A, int low, int high) {
    if (low == high) return A[low];
    int mid = (low + high) / 2;
    int left_max = maxSubarrayDC(A, low, mid);
    int right_max = maxSubarrayDC(A, mid + 1, high);
    int cross_max = maxCrossingSubarray(A, low, mid, high);
    return max({left_max, right_max, cross_max});
}

int maxSubarrayKadane(vector<int>& A) {
    int current_max = A[0];
    int global_max = A[0];
    for (int i = 1; i < A.size(); i++) {
        current_max = max(A[i], current_max + A[i]);
        global_max = max(global_max, current_max);
    }
    return global_max;
}
```

1. 把数组从中间分开，最大子数组要么全在左边，要么全在右边，要么跨过中点
2. 左边和右边的最大子数组：递归调用自己去找
3. 跨中点的最大子数组：从中点往左扫描找到最大左段，往右扫描找到最大右段，加起来
4. 三种情况的最大值就是答案
5. Kadane 算法更简单：遍历数组，对于每个位置决定”从这里重新开始”还是”延续之前的子数组”

4. 循环移位问题

给定一个按从小到大排序的数组，现将其循环右移若干位（如原数组 $[1,2,3,4,5]$ ，循环右移 2 位后为 $[4,5,1,2,3]$ ），设计算法计算循环右移的位数 k ($k \geq 0$)，要求时间复杂度尽可能优化。

- **核心性质**：循环右移后的数组是”两段有序子数组”的拼接，第一段所有元素 $\geq$ 第二段所有元素，循环右移位数 k 等于最小值元素的索引
- **分治查找**：类似折半搜索，比较 $A[\text{mid}]$ 与 $A[\text{high}]$ ，若 $A[\text{mid}] \leq A[\text{high}]$ 则右半有序，最小值在左半；否则最小值在右半
- **终止条件**： $\text{low} == \text{high}$ 时找到最小值，或当 $A[\text{mid}]$ 小于左右邻居时 mid 即为最小值位置
- **边界处理**：数组未移位（仍完全有序）时，最小值索引为 0， $k=0$
- **时间复杂度**：每次排除一半， $O(\log n)$

```
int findRotationCount(vector<int>& A) {
    int n = A.size();
    int low = 0, high = n - 1;
    if (A[low] <= A[high]) return 0;
    while (low < high) {
        int mid = (low + high) / 2;
        int prev = (mid - 1 + n) % n;
        int next = (mid + 1) % n;
        if (A[mid] <= A[prev] && A[mid] <= A[next]) {
            return mid;
        }
        if (A[mid] <= A[high]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
    return 0;
}
```

1. 先判断数组是否根本没移位（首元素 $\leq$ 尾元素），是的话直接返回 0
2. 找到中点，看它是不是最小值：比左边和右边都小，那就是了
3. 如果不是，判断哪半边是有序的：右半有序说明最小值在左半，左半有序说明最小值在右半
4. 向无序的那半边继续找（用二分的方式）
5. 找到最小值的位置就是循环右移的位数 k

5. 归并排序

给定数组 A，使用归并排序算法对其进行升序排序，要求展示分解 → 求解 → 合并”完整过程。

- **分解**：将数组按索引二分拆分为两个规模相等的子数组，重复拆分直至子数组长度为 1
- **基准情况**：子数组长度为 1 时本身已有序，直接作为子问题的解
- **合并**：对两个有序子数组用双指针法遍历，依次选取较小元素放入临时数组，合并为一个有序数组
- **递归结构**：先递归排序左半，再递归排序右半，最后合并左右
- **时间复杂度**：分解 $O(1)$ ，合并 $O(n)$ ，递归树深度 $\log n$ ，总计 $O(n \log n)$

```
void merge(vector<int>& A, int low, int mid, int high) {
    vector<int> L(A.begin() + low, A.begin() + mid + 1);
    vector<int> R(A.begin() + mid + 1, A.begin() + high + 1);
    int i = 0, j = 0, k = low;
    while (i < L.size() && j < R.size()) {
        if (L[i] <= R[j]) {
            A[k++] = L[i++];
        } else {
            A[k++] = R[j++];
        }
    }
    while (i < L.size()) A[k++] = L[i++];
    while (j < R.size()) A[k++] = R[j++];
}

void mergeSort(vector<int>& A, int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(A, low, mid);
        mergeSort(A, mid + 1, high);
        merge(A, low, mid, high);
    }
}
```

1. 把数组从中间分成两半
2. 对左半部分递归调用归并排序
3. 对右半部分递归调用归并排序
4. 合并左右两个有序部分：用两个指针分别指向左右部分的开头，每次选较小的放入原数组
5. 如果左边或右边还有剩余元素，直接全部复制过去
6. 重复上述步骤直到数组完全有序

6. 折半搜索（查找成功案例）

给定有序数组 A，使用折半搜索算法查找目标元素 target，要求展示每次查找的 low、mid、high 值及比较过程，若查找成功返回元素下标，失败返回-1。

- **初始化**：low=0（数组起始），high=n-1（数组结束）
- **中点计算**：mid= $\lfloor (low+high)/2 \rfloor$
- **比较调整**：若 target==A[mid] 则查找成功；若 target>A[mid] 则目标在右半，low=mid+1；若 target<A[mid] 则目标在左半，high=mid-1
- **终止条件**：找到目标返回 mid，或 low>high 时返回-1
- **时间复杂度**：每次排除一半， $O(\log n)$

```
int binarySearch(vector<int>& A, int target) {
    int low = 0, high = A.size() - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (A[mid] == target) {
            return mid;
        } else if (A[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}
```

1. 设置初始搜索范围：low 指向第一个元素，high 指向最后一个元素
2. 计算中点位置 mid，比较目标值和 A[mid] 的大小
3. 如果相等，找到了，返回 mid
4. 如果目标值更大，说明目标在右边，让 low 移到 mid+1
5. 如果目标值更小，说明目标在左边，让 high 移到 mid-1
6. 重复直到找到目标或 low 超过 high

7. 折半搜索（查找失败案例）

给定有序数组 A，使用折半搜索算法查找不存在的目标元素，重点关注查找失败时的边界调整及终止条件判断。

- **与成功案例的区别：** 算法流程完全一致，区别在于最终触发 low>high 的终止条件
- **关键观察：** 每次比较后缩小搜索范围，当 low>high 时说明搜索范围为空，目标不存在
- **边界动态：** low 和 high 逐步靠近，最终 low=high+1，表示没有元素可查
- **返回值：** 失败时返回-1（或其他约定值表示未找到）
- **时间复杂度：** 最坏情况需完全排除所有元素，O(log n)

```
int binarySearchRecursive(vector<int>& A, int target, int low, int high) {
    if (low > high) {
        return -1;
    }
    int mid = (low + high) / 2;
    if (A[mid] == target) {
        return mid;
    } else if (A[mid] < target) {
        return binarySearchRecursive(A, target, mid + 1, high);
    } else {
        return binarySearchRecursive(A, target, low, mid - 1);
    }
}

int binarySearch(vector<int>& A, int target) {
    int low = 0, high = A.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (A[mid] == target) return mid;
        else if (A[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

1. 初始化 low 和 high，确定搜索范围
2. 每次取中点 mid，比较目标值和 A[mid]
3. 根据比较结果调整 low 或 high，缩小搜索范围
4. 当 low>high 时，说明已经把所有可能的位置都查过了，没找到
5. 返回-1 表示查找失败

8. 大整数乘法

设 X 和 Y 为两个 n 位大整数（n 为偶数），X 拆分为 A（前 n/2 位）和 B（后 n/2 位），Y 拆分为 C（前 n/2 位）和 D（后 n/2 位），即 $X = A \cdot 2^{n/2} + B$ ， $Y = C \cdot 2^{n/2} + D$ 。设计优化方案通过减少乘法次数降低时间复杂度。

- **直接展开：** 需计算 AC、AD、BC、BD 共 4 次 n/2 位乘法，递归方程 $T(n) = 4T(n/2) + O(n)$ ，复杂度 $O(n^2)$ 未优化
- **Karatsuba 算法：** 通过代数变换设 $Z_1 = (A + B)(C + D) = AC + AD + BC + BD$ ，则 $XY = Z_1 \cdot 2^{n/2} + Z_2 \cdot 2^{n/2} + BD$ ，乘法次数降为 3 次
- **三次乘法：** 只需计算 $(A + B)(C + D)$ 、AC、BD，通过减法得到 $AD + BC = Z_1 - AC - BD$
- **递归方程：** $T(n) = 3T(n/2) + O(n)$ ，加法合并时间 O(n)
- **时间复杂度：** 由 Master 定理， $a = 3$ ， $b = 2$ ， $\log_2 a = \log_2 3 \approx 1.585 > 1$ ，故 $T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$

```
string karatsuba(string X, string Y) {
    int n = max(X.length(), Y.length());
    while (X.length() < n) X = "0" + X;
    while (Y.length() < n) Y = "0" + Y;
    if (n % 2 == 1) {
        n++;
        X = "0" + X;
        Y = "0" + Y;
    }
    if (n == 1) {
        int val = (X[0] - '0') * (Y[0] - '0');
        return to_string(val);
    }
    int m = n / 2;
    string A = X.substr(0, m);
    string B = X.substr(m);
    string C = Y.substr(0, m);
    string D = Y.substr(m);
    string Z1 = karatsuba(add(A, B), add(C, D));
    string Z2 = karatsuba(A, C);
    string Z3 = karatsuba(B, D);
    string Z4 = subtract(subtract(Z1, Z2), Z3);
    string result = add(shiftLeft(Z2, n), shiftLeft(Z4, m));
    result = add(result, Z3);
    return result;
}
```

1. 把两个大整数从中间对半拆，得到 A、B、C、D 四个部分
2. 直接展开需要 AC、AD、BC、BD 四次乘法，效率不高
3. 用 Karatsuba 技巧：先算 (A+B)(C+D)，再算 AC 和 BD，通过减法得到 AD+BC
4. 把三个结果按位权拼接：AC 左移 n 位，(AD+BC) 左移 n/2 位，BD 不用移
5. 把三部分加起来得到最终结果
6. 比传统方法少一次乘法，复杂度从 $O(n^2)$ 降到 $O(n^{1.585})$

9. Strassen 矩阵乘法

给定两个 $n \times n$ 矩阵 A 和 B，使用 Strassen 算法（分治法）计算矩阵乘积 $C=A \times B$ ，要求展示 7 个中间矩阵的定义及复杂度分析。

- **传统方法**: 按定义计算 C 的每个元素需 n 次乘法，共 n^3 次乘法，递归方程 $T(n) = 8T(n/2) + O(n^2)$
- **Strassen 优化**: 构造 7 个中间矩阵 M1 M7，通过 7 次乘法替代传统的 8 次乘法
- **7 个中间矩阵**:
 - $M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$, $M_2 = (A_{21} + A_{22})B_{11}$
 - $M_3 = A_{11}(B_{12} - B_{22})$, $M_4 = A_{22}(B_{21} - B_{11})$
 - $M_5 = (A_{11} + A_{12})B_{22}$, $M_6 = (A_{21} - A_{11})(B_{11} + B_{12})$
 - $M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$
- **合并结果**: $C_{11} = M_1 + M_4 - M_5 + M_7$, $C_{12} = M_3 + M_5$, $C_{21} = M_2 + M_4$, $C_{22} = M_1 - M_2 + M_3 + M_6$
- **时间复杂度**: $T(n) = 7T(n/2) + O(n^2)$, Master 定理得 $O(n^{\log_2 7}) \approx O(n^{2.807})$

```
vector<vector<int>> matrixAdd(vector<vector<int>>& A,
                             vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] + B[i][j];
    return C;}

vector<vector<int>> matrixSub(vector<vector<int>>& A,
                             vector<vector<int>>& B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n));
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            C[i][j] = A[i][j] - B[i][j];
    return C;}

vector<vector<int>> strassen(vector<vector<int>>& A,
                             vector<vector<int>>& B) {
    int n = A.size();
    if (n <= 64) {
        vector<vector<int>> C(n, vector<int>(n, 0));
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                for (int k = 0; k < n; k++)
                    C[i][j] += A[i][k] * B[k][j];
        return C;
    }
    int m = n / 2;
    return C;}
```

1. 把大矩阵从中间分成 4 个小矩阵: A11、A12、A21、A22 和 B 的对应 4 块
2. 传统方法需要 8 次小矩阵乘法才能得到 C 的 4 个块
3. Strassen 方法计算 7 个特殊的中间矩阵 M1 到 M7，每个都是小矩阵的加减后再乘
4. 通过这 7 个中间矩阵的加减运算，拼出 C 的 4 个块
5. 虽然加减次数增多，但乘法从 8 次减到 7 次，递归下去效率提升明显
6. 复杂度从 $O(n^3)$ 降到 $O(n^{2.807})$ ，矩阵大时优势显著

10. 快速排序

给定数组 A，使用快速排序算法对其进行升序排序，要求展示 Partition 过程及完整的递归排序步骤。

- **Partition 过程**: 选择基准元素 pivot（常选首元素），维护指针 p 指向小于 pivot 的区域的末尾，遍历数组使小于 pivot 的元素交换到前面
- **分区逻辑**: i 从 1 到 n-1 遍历，若 $A[i] < pivot$ 则 $p++$ ，交换 $A[p]$ 与 $A[i]$ ，最后交换 $A[0]$ 与 $A[p]$ 使 pivot 归位
- **递归排序**: 对 pivot 左侧子数组 $A[0..p-1]$ 和右侧子数组 $A[p+1..n-1]$ 分别递归调用快排
- **终止条件**: 子数组长度为 0 或 1 时已有序，直接返回
- **时间复杂度**: 平均 $O(n \log n)$ （每次平衡划分），最坏 $O(n^2)$ （每次划分极不平衡）

```
int partition(vector<int>& A, int low, int high) {
    int pivot = A[low];
    int p = low;
    for (int i = low + 1; i <= high; i++) {
        if (A[i] < pivot) {
            p++;
            swap(A[p], A[i]);
        }
        swap(A[low], A[p]);
        return p;
    }
}

void quickSort(vector<int>& A, int low, int high) {
    if (low < high) {
        int p = partition(A, low, high);
        quickSort(A, low, p - 1);
        quickSort(A, p + 1, high);
    }
}
```

1. 选第一个元素作为基准，用指针 p 记录”小于基准的区域”的右边界
2. 从第二个元素开始遍历，如果比基准小，就扩展 p，把这个元素交换到前面的区域
3. 遍历完后，把基准元素交换到 p 位置，这样基准左边都小，右边都大（或相等）
4. 对基准左边的子数组递归调用快排
5. 对基准右边的子数组递归调用快排
6. 重复直到所有子数组长度不超过 1，排序完成

11. 第 k 小元素（选择问题）

给定无序数组 A，使用分治法查找该数组的第 k 小元素，要求展示 Partition 过程及递归查找步骤。

- **Partition 分区**：选择基准元素 pivot，划分数组为左（小于 pivot）、中（pivot）、右（大于等于 pivot）三部分，返回 pivot 的最终位置 pos
- **k 值判断**：若 $k == pos + 1$ 则 pivot 即为第 k 小元素；若 $k < pos + 1$ 则在左子数组递归查找第 k 小；若 $k > pos + 1$ 则在右子数组递归查找第 k-pos-1 小
- **k 值调整**：递归右子数组时，k 值需减去 pos+1（排除左子数组和 pivot）
- **平均复杂度**：每次 Partition 期望划分平衡， $T(n) = T(n/2) + O(n)$ ，得 $O(n)$ ；最坏 $O(n^2)$ （每次划分极不平衡）
- **优化**：随机选择 pivot 或使用 Median-of-Medians 算法可保证最坏 $O(n)$

```
int partition(vector<int>& A, int low, int high) {
    int pivot = A[low];
    int p = low;
    for (int i = low + 1; i <= high; i++) {
        if (A[i] < pivot) {
            p++;
            swap(A[p], A[i]);
        }
    }
    swap(A[low], A[p]);
    return p;
}

int quickSelect(vector<int>& A, int low, int high, int k) {
    if (low == high) return A[low];
    int pos = partition(A, low, high);
    if (k == pos + 1) {
        return A[pos];
    } else if (k < pos + 1) {
        return quickSelect(A, low, pos - 1, k);
    } else {
        return quickSelect(A, pos + 1, high, k - pos - 1);
    }
}

int findKthSmallest(vector<int>& A, int k) {
    return quickSelect(A, 0, A.size() - 1, k);
}
```

1. 选一个基准元素，用 Partition 把数组分成“小于基准”和“大于等于基准”两部分
2. 基准元素归位后得到位置 pos，说明它左边有 pos 个元素比它小
3. 如果 k 等于 pos+1，那基准就是第 k 小元素，直接返回
4. 如果 k 小于 pos+1，说明第 k 小元素在左边，递归在左边找第 k 小
5. 如果 k 大于 pos+1，说明第 k 小元素在右边，递归在右边找第 k-pos-1 小（要减去左边和基准）
6. 重复直到找到为止

12. 堆排序

给定数组 A，使用堆排序算法对其进行升序排序，要求展示最大堆的建立过程、堆顶元素的弹出与堆调整过程。

- **堆性质**：最大堆中每个节点值 $\geq$ 其子节点值，数组表示中节点 i 的左子为 $2i+1$ ，右子为 $2i+2$ ，父为 $\lfloor (i-1)/2 \rfloor$
- **建堆**：从最后一个非叶子节点 $\lfloor n/2 \rfloor - 1$ 开始，向前依次执行 Heapify 向下调整，时间 $O(n)$
- **Heapify**：对节点 i，若其值小于子节点，则与最大子节点交换，并递归调整被交换的子节点
- **排序循环**：交换堆顶（最大值）与堆尾，缩小堆范围，对新的堆顶执行 Heapify 重建最大堆
- **时间复杂度**：建堆 $O(n)$ ，n 次弹出各 $O(\log n)$ ，总计 $O(n \log n)$ ；空间 $O(1)$ 原地排序

```
void heapify(vector<int>& A, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;
    if (left < n && A[left] > A[largest])
        largest = left;
    if (right < n && A[right] > A[largest])
        largest = right;
    if (largest != i) {
        swap(A[i], A[largest]);
        heapify(A, n, largest);
    }
}

void buildHeap(vector<int>& A, int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(A, n, i);
    }
}

void heapSort(vector<int>& A) {
    int n = A.size();
    buildHeap(A, n);
    for (int i = n - 1; i > 0; i--) {
        swap(A[0], A[i]);
        heapify(A, i, 0);
    }
}
```

1. 从最后一个非叶子节点开始，对每个节点执行 Heapify 调整，把数组变成最大堆
2. Heapify 调整时，如果节点比子节点小，就跟最大子节点交换，然后继续往下调整
3. 建好堆后，堆顶就是最大值，把它和堆尾元素交换，最大值就放到有序区了
4. 堆范围缩小 1，对新的堆顶再执行 Heapify 重建最大堆
5. 重复“交换堆顶和堆尾、缩小堆、重建堆”这个过程
6. 直到堆只剩一个元素，排序完成

13. 最近点对问题

给定平面上 n 个点的集合 Q，设计分治算法找出距离最近的两个点，要求说明预处理、分解、递归求解、合并的完整流程。

- **预处理:** 将点集按 x 坐标排序，便于均匀划分，时间 $O(n \log n)$
- **分解:** 按 x 坐标中位数用垂线 $x=m$ 划分点集为 L ($x \leq m$) 和 R ($x > m$)，确保规模平衡
- **递归求解:** 分别找 L 和 R 的最近点对，距离为 d_1 和 d_2 ，令 $d = \min(d_1, d_2)$
- **合并 (临界区):** 筛选 $x \in [m-d, m+d]$ 的点集 Yd 按 y 排序，对每个点仅与后 6 个点比较 (鸽巢原理)，时间 $O(n)$
- **时间复杂度:** 递归方程 $T(n) = 2T(n/2) + O(n)$ ，Master 定理得 $O(n \log n)$

```
struct Point {
    double x, y;
double dist(Point p1, Point p2) {
    return sqrt((p1.x - p2.x) * (p1.x - p2.x) +
                (p1.y - p2.y) * (p1.y - p2.y));}
double bruteForce(vector<Point>& pts, int left, int right) {
    double minDist = INT_MAX;
    for (int i = left; i <= right; i++)
        for (int j = i + 1; j <= right; j++)
            minDist = min(minDist, dist(pts[i], pts[j]));
    return minDist;}
double stripClosest(vector<Point>& strip, double d) {
    double minDist = d;
    sort(strip.begin(), strip.end(),
         [](Point a, Point b) { return a.y < b.y; });
    for (size_t i = 0; i < strip.size(); i++) {
        for (size_t j = i + 1; j < strip.size() &&
             (strip[j].y - strip[i].y) < minDist; j++) {
            minDist = min(minDist, dist(strip[i], strip[j]));
        }
    }
    return minDist;}
double closestUtil(vector<Point>& pts, int left, int right) {
    if (right - left + 1 <= 3)
        return bruteForce(pts, left, right);
    int mid = (left + right) / 2;
    Point midPoint = pts[mid];
    double d1 = closestUtil(pts, left, mid);
    double d2 = closestUtil(pts, mid + 1, right);
    double d = min(d1, d2);
    vector<Point> strip;
    for (int i = left; i <= right; i++)
        if (abs(pts[i].x - midPoint.x) < d)
            strip.push_back(pts[i]);
    return min(d, stripClosest(strip, d));}}}
double closestPair(vector<Point>& pts) {
    sort(pts.begin(), pts.end(),
         [](Point a, Point b) { return a.x < b.x; });
    return closestUtil(pts, 0, pts.size() - 1);}
```

1. 先把所有点按 x 坐标排序，方便后面均匀划分
2. 从 x 坐标中位数位置把点集分成左右两半，递归在两边找最近的点对
3. 得到两边的最小距离 d 后，最近点对要么在左边，要么在右边，要么跨过分界线
4. 跨界点对只能在分界线左右各 d 宽度的条形区域里
5. 把这个条形区域的点按 y 坐标排序，每个点只需跟后面 6 个点比较距离 (鸽巢原理保证)
6. 三种情况 (左、右、跨) 取最小就是最终答案

14. 天际轮廓问题

给定 n 座建筑物，每个建筑物用三元组 (Li,Ri,Hi) 表示 (Li 为左下 x 坐标，Ri 为右下 x 坐标，Hi 为高度)，设计 $O(n \log n)$ 算法求出这 n 座建筑物的天际轮廓。

- **天际轮廓表示:** 用高度变化的关键点序列表示，如单个建筑物的轮廓为 [(Li,0),(Li,Hi),(Ri,Hi),(Ri,0)]
- **分治分解:** 将建筑物集均分为左右两个子集，递归求得左右
- **双指针合并:** 用双指针 i、j 分别遍历左右天际线，按 x 坐标升序合并，同时维护当前左右高度 hL、hR，取 $\max(hL, hR)$ 作为合并后当前高度
- **关键点记录:** 当当前 $\max(hL, hR)$ 与上次记录的高度不同时，记录新的关键点 (x, $\max(hL, hR)$)
- **时间复杂度** $T(n) = 2T(n/2) + O(n)$ ，得 $O(n \log n)$

```
struct Point {
    int x, height;
    bool isStart;
};
vector<Point> mergeSkylines(vector<Point>& left, vector<Point>& right) {
    vector<Point> result;
    int i = 0, j = 0;
    int hL = 0, hR = 0, currH = 0;
    while (i < left.size() && j < right.size()) {
        int x;
        if (left[i].x < right[j].x ||
            (left[i].x == right[j].x && left[i].height >= right[j].height)) {
            hL = left[i].height;
            x = left[i].x;
            i++;
        } else {
            hR = right[j].height;
            x = right[j].x;
            j++;
        }
        int newH = max(hL, hR);
        if (result.empty() || newH != currH) {
            result.push_back({x, newH});
            currH = newH;
        }
        while (i < left.size()) {
            if (left[i].height != currH) {
                result.push_back(left[i]);
                currH = left[i].height;
            }
            i++;
        }
        while (j < right.size()) {
            if (right[j].height != currH) {
                result.push_back(right[j]);
                currH = right[j].height;
            }
            j++;
        }
        return result;
    }
vector<Point> getSkyline(vector<vector<int>>& buildings, int low, int high) {
    if (low == high) {
        return {{buildings[low][0], buildings[low][2]},
                {buildings[low][1], 0}};
    }
    int mid = (low + high) / 2;
    auto left = getSkyline(buildings, low, mid);
    auto right = getSkyline(buildings, mid + 1, high);
    return mergeSkylines(left, right);}
```

1. 如果只有一个建筑，天际线就是它的 4 个顶点
2. 多个建筑时，从中间分成左右两半，递归求各自的天际线
3. 用双指针遍历两条天际线，x 坐标小的先处理
4. 维护当前左右天际线的高度，取最大值为合并后当前高度
5. 只要当前最大高度和上次记录不一样，就记录新的关键点
6. 重复直到处理完所有关键点，最终得到的就是完整天际线

15. 两元素和为 X 的分治算法

给定由 n 个实数构成的集合 S 和实数 X ，判断 S 中是否存在两个元素的和为 X ，要求设计分治算法求解。

- **预处理**：先用归并排序对 S 排序，时间 $O(n \log n)$ ，为后续双指针查找做准备
- **分治分解**：将有序集合 S 均分为左子集 L 和右子集 R
- **递归求解**：递归判断 L 和 R 内部是否存在两元素和为 X
- **合并验证**：用双指针 i （指向 L 首）、 j （指向 R 尾）遍历，判断是否存在跨子集的两元素和为 X
- **双指针逻辑**：若 $S[i] + S[j] == X$ 则返回 true；若 $< X$ 则 $i++$ （增大和）；若 $> X$ 则 $j--$ （减小和）
- **时间复杂度**：排序 $O(n \log n)$ ，递归方程 $T(n) = 2T(n/2) + O(n)$ ，总计 $O(n \log n)$

```
bool hasPairCross(vector<int>& S, int low, int mid, int high, int X) {
    int i = low;
    int j = high;
    while (i <= mid && j > mid) {
        int sum = S[i] + S[j];
        if (sum == X) {
            return true;
        } else if (sum < X) {
            i++;
        } else {
            j--;
        }
    }
    return false;
}

bool hasTwoSumDC(vector<int>& S, int low, int high, int X) {
    if (low >= high) return false;
    int mid = (low + high) / 2;
    bool leftFound = hasTwoSumDC(S, low, mid, X);
    bool rightFound = hasTwoSumDC(S, mid + 1, high, X);
    if (leftFound || rightFound) return true;
    return hasPairCross(S, low, mid, high, X);
}

bool hasTwoSum(vector<int>& S, int X) {
    sort(S.begin(), S.end());
    return hasTwoSumDC(S, 0, S.size() - 1, X);
}
```

1. 先把数组排序（用归并排序），为双指针查找做准备
2. 把数组从中间分成左右两半，递归判断两边内部有没有符合条件的
3. 对于跨两边的情况，用双指针：一个指左半开头，一个指右半结尾
4. 计算两个指针指向的元素的和，如果等于 X 就找到了
5. 如果和小于 X ，左指针右移（找个更大的）；如果大于 X ，右指针左移（找个更小的）
6. 三种情况（左内、右内、跨）只要有一个 true 就返回 true

16. 逆序对的分治算法

给定实数序列 A ，若 $i < j$ 且 $A[i] > A[j]$ ，则 (i, j) 为逆序对。要求用分治方法求序列中逆序对的个数。

- **逆序对定义**：前大后小的元素对，如 $[3, 1, 2]$ 的逆序对为 $(3, 1)$ 、 $(3, 2)$
- **分治分解**：将序列均分为左子序列 L 和右子序列 R
- **递归求解**：递归计算 L 和 R 内部的逆序对个数 countL 和 countR
- **合并统计**：归并 L 和 R 为有序序列，同时统计跨 L 、 R 的逆序对个数 cross
- **跨区间统计**：归并时若 $L[i] > R[j]$ ，则 $L[i]$ 及 $L[i]$ 之后的所有元素都与 $R[j]$ 构成逆序对， $\text{count} += \text{mid} - i + 1$
- **时间复杂度**：归并排序的合并步骤嵌入统计， $T(n) = 2T(n/2) + O(n)$ ，总计 $O(n \log n)$

```
int mergeAndCount(vector<int>& A, int low, int mid, int high) {
    vector<int> L(A.begin() + low, A.begin() + mid + 1);
    vector<int> R(A.begin() + mid + 1, A.begin() + high + 1);
    int i = 0, j = 0, k = low;
    int count = 0;
    while (i < L.size() && j < R.size()) {
        if (L[i] <= R[j]) {
            A[k++] = L[i++];
        } else {
            A[k++] = R[j++];
            count += L.size() - i;
        }
    }
    while (i < L.size()) A[k++] = L[i++];
    while (j < R.size()) A[k++] = R[j++];
    return count;
}

int countInversions(vector<int>& A, int low, int high) {
    if (low >= high) return 0;
    int mid = (low + high) / 2;
    int countL = countInversions(A, low, mid);
    int countR = countInversions(A, mid + 1, high);
    int countCross = mergeAndCount(A, low, mid, high);
    return countL + countR + countCross;
}

int countInversions(vector<int>& A) {
    return countInversions(A, 0, A.size() - 1);
}
```

1. 把序列从中间分成左右两半
2. 递归统计左半内部的逆序对个数
3. 递归统计右半内部的逆序对个数
4. 归并左右两半为有序序列，同时统计跨区间的逆序对
5. 统计跨区间时，如果左边元素大于右边，那左边剩下的元素也都大于右边当前元素
6. 总逆序对数等于“左内 + 右内 + 跨区间”之和

17. 数组中两元素和大于 0 的对数量

给定大小为 n 的数组 A ，求数组中不同对 (i, j) ($i < j$) 的数量，使得 $A[i] + A[j] > 0$ 。

- **分治排序**：用归并排序将 A 升序排列，时间 $O(n \log n)$ ，为双指针统计做准备
- **分治分解**：将有序数组划分为左子集 L 和右子集 R
- **递归统计**：分别计算 L 内部、 R 内部符合条件的对数量 countL 和 countR
- **合并统计**：用双指针 i （指向 L 首）、 j （指向 R 尾）遍历，统计跨子集的对数量 cross
- **双指针逻辑**：若 $A[i] + A[j] > 0$ ，则 $R[j]$ 及 $R[j]$ 之前的所有元素与 $A[i]$ 的和都 > 0 ， $\text{count} += j - \text{mid} + 1$ ， $i++$ ；否则 $j--$
- **时间复杂度**：排序 $O(n \log n)$ ，分治递归方程 $T(n) = 2T(n/2) + O(n)$ ，总计 $O(n \log n)$

```
int countCross(vector<int>& A, int low, int mid, int high) {
    int i = low;
    int j = high;
    int count = 0;
    while (i <= mid && j > mid) {
        if (A[i] + A[j] > 0) {
            count += j - mid;
            i++;
        } else {
            j--;
        }
    }
    return count;
}

int countPairsDC(vector<int>& A, int low, int high) {
    if (low >= high) return 0;
    int mid = (low + high) / 2;
    int countL = countPairsDC(A, low, mid);
    int countR = countPairsDC(A, mid + 1, high);
    int countCross = countCross(A, low, mid, high);
    return countL + countR + countCross;
}

int countPairsWithSumGreaterThanZero(vector<int>& A) {
    sort(A.begin(), A.end());
    return countPairsDC(A, 0, A.size() - 1);
}
```

1. 先用归并排序把数组升序排列
2. 把数组从中间分成左右两半，递归统计两边内部符合条件的对数
3. 对于跨两边的情况，用双指针：一个指左半开头，一个指右半结尾
4. 如果两个指针指向的元素和大于 0，那右半剩下的所有元素跟左半当前元素的和也都大于 0
5. 累加符合条件的对数，然后左指针右移（尝试更小的元素）
6. 如果和小于等于 0，右指针左移（尝试更大的元素）
7. 总对数等于“左内 + 右内 + 跨子集”之和

分治算法解题的”固有方法论”

分治算法（Divide and Conquer）的核心思想是”大事化小，小事化了，最后合并”。在解决此类题目时，可以遵循标准的”三步走”策略：

1. 分解 (Divide)：将原问题划分成若干个规模较小、相互独立、与原问题形式相同的子问题。

• 代码体现：通常是寻找中点 mid ，将数组下标范围划分为 $[\text{left}, \text{mid}]$ 和 $[\text{mid}+1, \text{right}]$ 。

2. 解决 (Conquer)：递归地求解这些子问题。

• 基准情况 (Base Case)：必须定义递归结束的条件（例如数组长度为 1 或 2 时），直接求解。

• 代码体现：递归调用函数本身，如 $\text{solve}(\text{left}, \text{mid})$ 。

3. 合并 (Combine)：将子问题的解合并，得到原问题的解。这是分治算法中最关键、最难设计的一步。

• 代码体现：在递归返回后，编写逻辑将左右两边的结果整合成一个结果。

复杂度分析套路：

通常使用主定理（Master Theorem）或递归树。形式为 $T(n) = aT(n/b) + f(n)$ 。

- a ：子问题个数
- b ：子问题缩小的倍数
- $f(n)$ ：合并步骤的代价